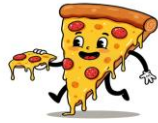


PRACTICE ASSESSMENT 2

(SQL Basic Commands)

Pizza Palace



Sample Tables Setup

```
CREATE DATABASE pizza;
```

```
USE pizza;
```

```
CREATE TABLE pizzas (  
  pizza_id INT PRIMARY KEY,  
  pizza_name VARCHAR(50),  
  size ENUM('Small', 'Medium', 'Large', 'Extra Large'),  
  crust_type VARCHAR(30),  
  price DECIMAL(6,2),  
  calories INT,  
  category VARCHAR(20),  
  is_vegetarian BOOLEAN  
);
```

```
INSERT INTO pizzas VALUES  
(1, 'Margherita', 'Medium', 'Thin Crust', 12.99, 280, 'Classic', TRUE),  
(2, 'Pepperoni Supreme', 'Large', 'Hand Tossed', 18.99, 350, 'Meat', FALSE),  
(3, 'Hawaiian Paradise', 'Medium', 'Stuffed Crust', 15.99, 320, 'Specialty', FALSE),  
(4, 'Veggie Delight', 'Small', 'Thin Crust', 10.99, 220, 'Vegetarian', TRUE),  
(5, 'Meat Lovers', 'Extra Large', 'Thick Crust', 22.99, 450, 'Meat', FALSE),  
(6, 'BBQ Chicken', 'Large', 'Hand Tossed', 19.99, 380, 'Specialty', FALSE),
```

(7, 'Mushroom Magic', 'Medium', 'Thin Crust', 13.99, 260, 'Vegetarian', TRUE),
(8, 'Spicy Italian', 'Large', 'Stuffed Crust', 20.99, 400, 'Specialty', FALSE),
(9, 'Four Cheese', 'Small', 'Hand Tossed', 11.99, 310, 'Classic', TRUE),
(10, 'Buffalo Chicken', 'Medium', 'Thick Crust', 16.99, 340, 'Specialty', FALSE),
(11, 'Garden Fresh', 'Large', 'Thin Crust', 17.99, 290, 'Vegetarian', TRUE),
(12, 'Supreme Deluxe', 'Extra Large', 'Hand Tossed', 24.99, 420, 'Specialty', FALSE);

```
CREATE TABLE customers (  
customer_id INT PRIMARY KEY,  
first_name VARCHAR(30),  
last_name VARCHAR(30),  
phone VARCHAR(15),  
email VARCHAR(60),  
city VARCHAR(30),  
join_date DATE,  
loyalty_points INT,  
status ENUM('Regular', 'VIP', 'Gold', 'Inactive')  
);
```

```
INSERT INTO customers VALUES  
(201, 'Tony', 'Soprano', '555-0101', 'tony.soprano@email.com', 'Newark', '2023-01-15', 450,  
'VIP'),  
(202, 'Maria', 'Rossi', '555-0102', 'maria.rossi@email.com', 'Brooklyn', '2023-02-20', 280,  
'Regular'),  
(203, 'Giuseppe', 'Bianchi', '555-0103', 'giuseppe.b@email.com', 'Manhattan', '2022-11-10', 750,  
'Gold'),  
(204, 'Isabella', 'Ferrari', '555-0104', 'isabella.f@email.com', 'Queens', '2023-03-05', 120,  
'Regular'),
```

(205, 'Marco', 'Lombardi', '555-0105', 'marco.lombardi@email.com', 'Bronx', '2022-08-18', 890, 'Gold'),
(206, 'Sofia', 'Romano', '555-0106', 'sofia.romano@email.com', 'Brooklyn', '2023-04-12', 65, 'Regular'),
(207, 'Antonio', 'Conti', '555-0107', 'antonio.conti@email.com', 'Staten Island', '2022-12-03', 340, 'VIP'),
(208, 'Francesca', 'Moretti', '555-0108', 'francesca.m@email.com', 'Manhattan', '2023-01-28', 180, 'Regular'),
(209, 'Luca', 'Esposito', '555-0109', 'luca.esposito@email.com', 'Queens', '2022-09-15', 520, 'VIP'),
(210, 'Giulia', 'Ricci', '555-0110', 'giulia.ricci@email.com', 'Bronx', '2023-05-22', 15, 'Inactive');

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  delivery_time TIME,  
  total_amount DECIMAL(8,2),  
  delivery_type ENUM('Pickup', 'Delivery', 'Dine-in'),  
  order_status ENUM('Pending', 'Preparing', 'Ready', 'Delivered', 'Cancelled'),  
  special_instructions TEXT  
);
```

```
INSERT INTO orders VALUES  
(301, 201, '2024-01-15', '18:30:00', 45.97, 'Delivery', 'Delivered', 'Extra cheese on all pizzas'),  
(302, 202, '2024-01-16', '12:15:00', 23.98, 'Pickup', 'Ready', 'No onions please'),  
(303, 203, '2024-01-16', '19:45:00', 67.95, 'Dine-in', 'Delivered', NULL),  
(304, 204, '2024-01-17', '20:00:00', 32.99, 'Delivery', 'Preparing', 'Call before delivery'),
```

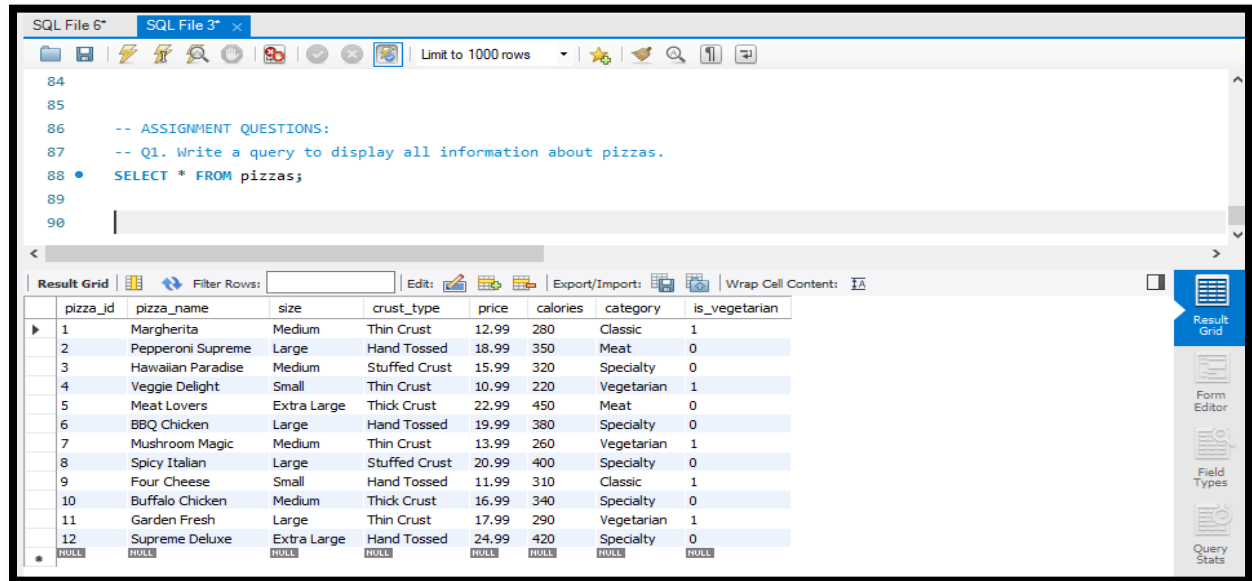
(305, 205, '2024-01-17', '13:30:00', 18.99, 'Pickup', 'Ready', 'Extra spicy'),
(306, 206, '2024-01-18', '17:15:00', 28.98, 'Delivery', 'Delivered', NULL),
(307, 207, '2024-01-18', '19:00:00', 41.97, 'Dine-in', 'Cancelled', 'Table for 4'),
(308, 208, '2024-01-19', '14:45:00', 15.99, 'Pickup', 'Delivered', 'Light sauce'),
(309, 209, '2024-01-19', '21:30:00', 52.96, 'Delivery', 'Delivered', 'Ring doorbell twice'),
(310, 210, '2024-01-20', '16:20:00', 12.99, 'Pickup', 'Pending', NULL);

ASSESSMENT QUESTIONS:

Section A: Basic SELECT Statements

Q1. Write a query to display all information about pizzas.

SELECT * FROM pizzas;

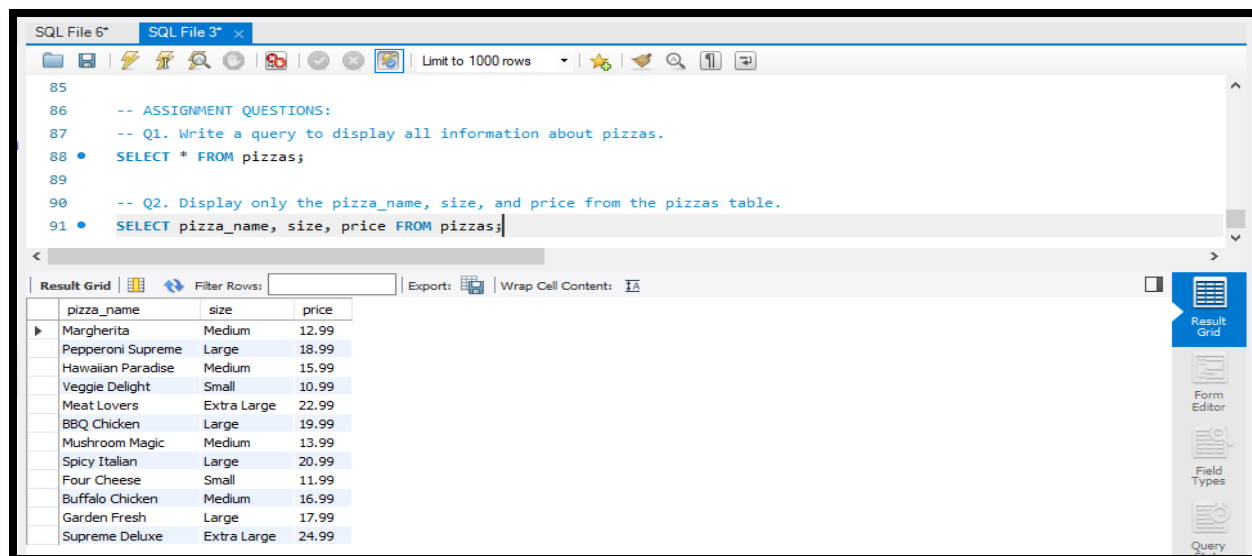


The screenshot shows a SQL editor window titled 'SQL File 6*'. The query entered is:
-- ASSIGNMENT QUESTIONS:
-- Q1. Write a query to display all information about pizzas.
SELECT * FROM pizzas;
The 'Result Grid' below the query displays the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
3	Hawaiian Paradise	Medium	Stuffed Crust	15.99	320	Specialty	0
4	Veggie Delight	Small	Thin Crust	10.99	220	Vegetarian	1
5	Meat Lovers	Extra Large	Thick Crust	22.99	450	Meat	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
8	Spicy Italian	Large	Stuffed Crust	20.99	400	Specialty	0
9	Four Cheese	Small	Hand Tossed	11.99	310	Classic	1
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
12	Supreme Deluxe	Extra Large	Hand Tossed	24.99	420	Specialty	0

Q2. Display only the pizza_name, size, and price from the pizzas table.

SELECT pizza_name, size, price FROM pizzas;

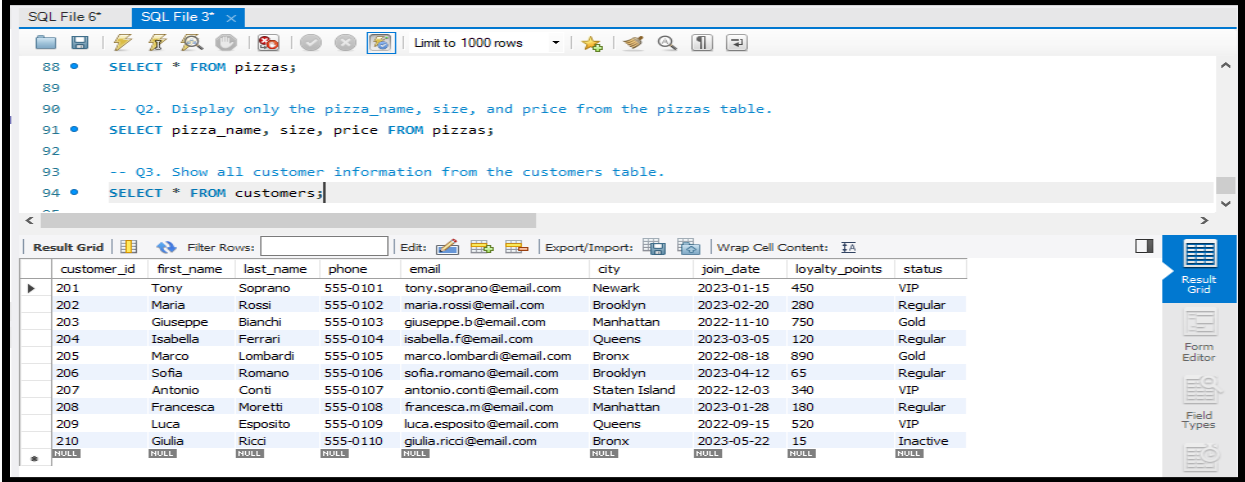


The screenshot shows a SQL editor window titled 'SQL File 3*'. The query entered is:
-- ASSIGNMENT QUESTIONS:
-- Q1. Write a query to display all information about pizzas.
SELECT * FROM pizzas;
-- Q2. Display only the pizza_name, size, and price from the pizzas table.
SELECT pizza_name, size, price FROM pizzas;
The 'Result Grid' below the query displays the following data:

pizza_name	size	price
Margherita	Medium	12.99
Pepperoni Supreme	Large	18.99
Hawaiian Paradise	Medium	15.99
Veggie Delight	Small	10.99
Meat Lovers	Extra Large	22.99
BBQ Chicken	Large	19.99
Mushroom Magic	Medium	13.99
Spicy Italian	Large	20.99
Four Cheese	Small	11.99
Buffalo Chicken	Medium	16.99
Garden Fresh	Large	17.99
Supreme Deluxe	Extra Large	24.99

Q3. Show all customer information from the customers table.

SELECT * FROM customers;

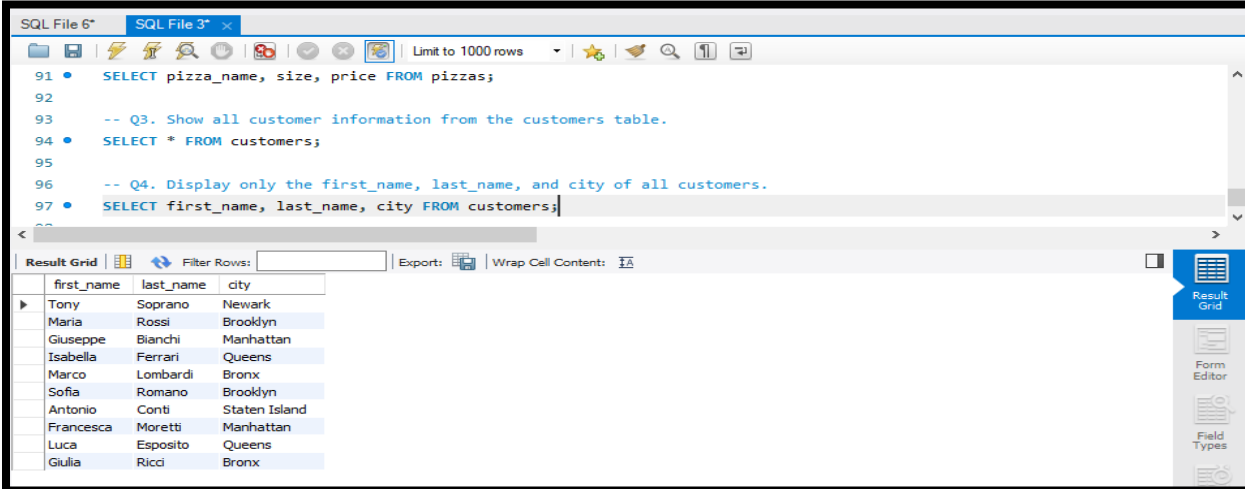


```
88 • SELECT * FROM pizzas;
89
90 -- Q2. Display only the pizza_name, size, and price from the pizzas table.
91 • SELECT pizza_name, size, price FROM pizzas;
92
93 -- Q3. Show all customer information from the customers table.
94 • SELECT * FROM customers;
```

	customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
▶	201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
	202	Maria	Rossi	555-0102	maria.rossi@email.com	Brooklyn	2023-02-20	280	Regular
	203	Giuseppe	Bianchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
	204	Isabella	Ferrari	555-0104	isabella.f@email.com	Queens	2023-03-05	120	Regular
	205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
	206	Sofia	Romano	555-0106	sofia.romano@email.com	Brooklyn	2023-04-12	65	Regular
	207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP
	208	Francesca	Moretti	555-0108	francesca.m@email.com	Manhattan	2023-01-28	180	Regular
	209	Luca	Esposito	555-0109	luca.esposito@email.com	Queens	2022-09-15	520	VIP
	210	Giulia	Ricci	555-0110	giulia.ricci@email.com	Bronx	2023-05-22	15	Inactive
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q4. Display only the first_name, last_name, and city of all customers.

SELECT first_name, last_name, city FROM customers;



```
91 • SELECT pizza_name, size, price FROM pizzas;
92
93 -- Q3. Show all customer information from the customers table.
94 • SELECT * FROM customers;
95
96 -- Q4. Display only the first_name, last_name, and city of all customers.
97 • SELECT first_name, last_name, city FROM customers;
```

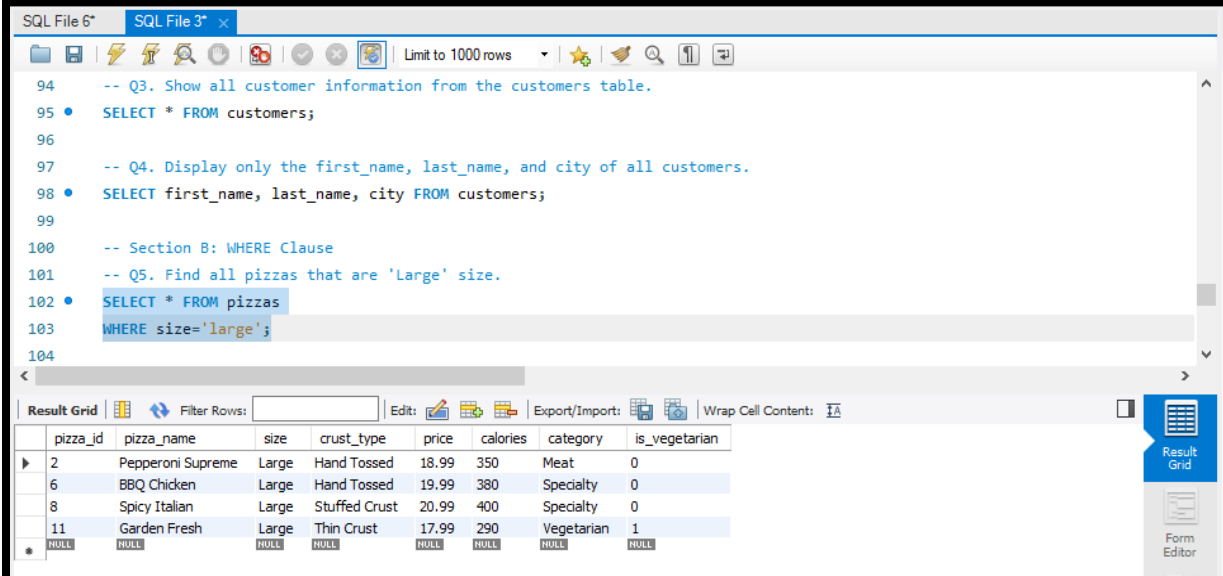
	first_name	last_name	city
▶	Tony	Soprano	Newark
	Maria	Rossi	Brooklyn
	Giuseppe	Bianchi	Manhattan
	Isabella	Ferrari	Queens
	Marco	Lombardi	Bronx
	Sofia	Romano	Brooklyn
	Antonio	Conti	Staten Island
	Francesca	Moretti	Manhattan
	Luca	Esposito	Queens
	Giulia	Ricci	Bronx

Section B: WHERE Clause

Q5. Find all pizzas that are 'Large' size.

```
SELECT * FROM pizzas
```

```
WHERE size='large';
```



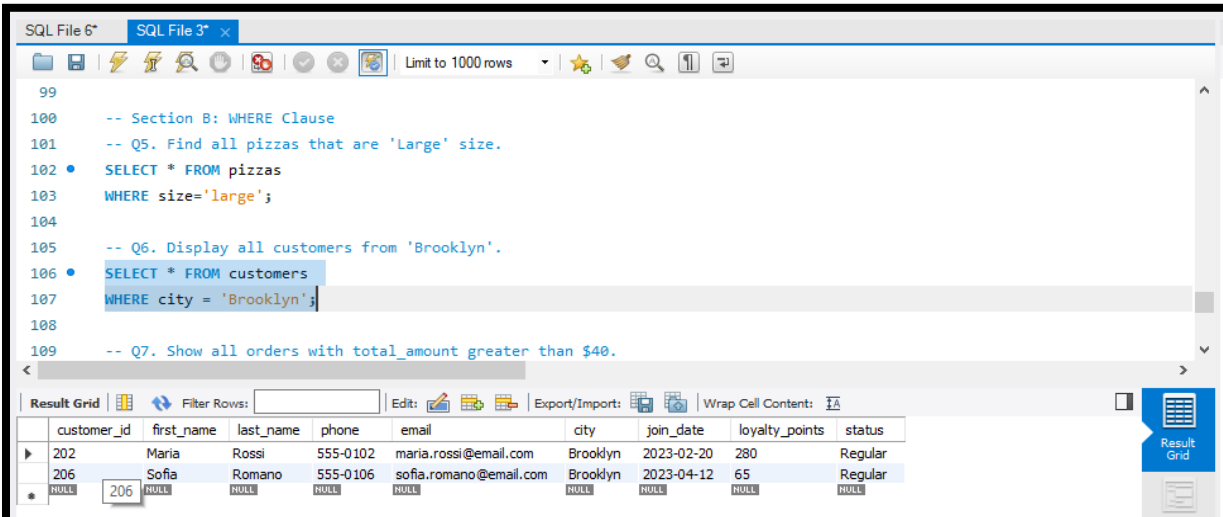
The screenshot shows a SQL editor window titled 'SQL File 3*' with a toolbar and a 'Limit to 1000 rows' dropdown. The SQL code in the editor includes comments for Q3, Q4, and Q5, and the query for Q5 is highlighted: `SELECT * FROM pizzas WHERE size='large';`. Below the editor is a 'Result Grid' showing the results of the query. The grid has 8 columns: `pizza_id`, `pizza_name`, `size`, `crust_type`, `price`, `calories`, `category`, and `is_vegetarian`. The results show four rows of data for large pizzas.

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0
8	Spicy Italian	Large	Stuffed Crust	20.99	400	Specialty	0
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q6. Display all customers from 'Brooklyn'.

```
SELECT * FROM customers
```

```
WHERE city = 'Brooklyn';
```



The screenshot shows a SQL editor window titled 'SQL File 3*' with a toolbar and a 'Limit to 1000 rows' dropdown. The SQL code in the editor includes comments for Q5, Q6, and Q7, and the query for Q6 is highlighted: `SELECT * FROM customers WHERE city = 'Brooklyn';`. Below the editor is a 'Result Grid' showing the results of the query. The grid has 9 columns: `customer_id`, `first_name`, `last_name`, `phone`, `email`, `city`, `join_date`, `loyalty_points`, and `status`. The results show three rows of data for customers from Brooklyn.

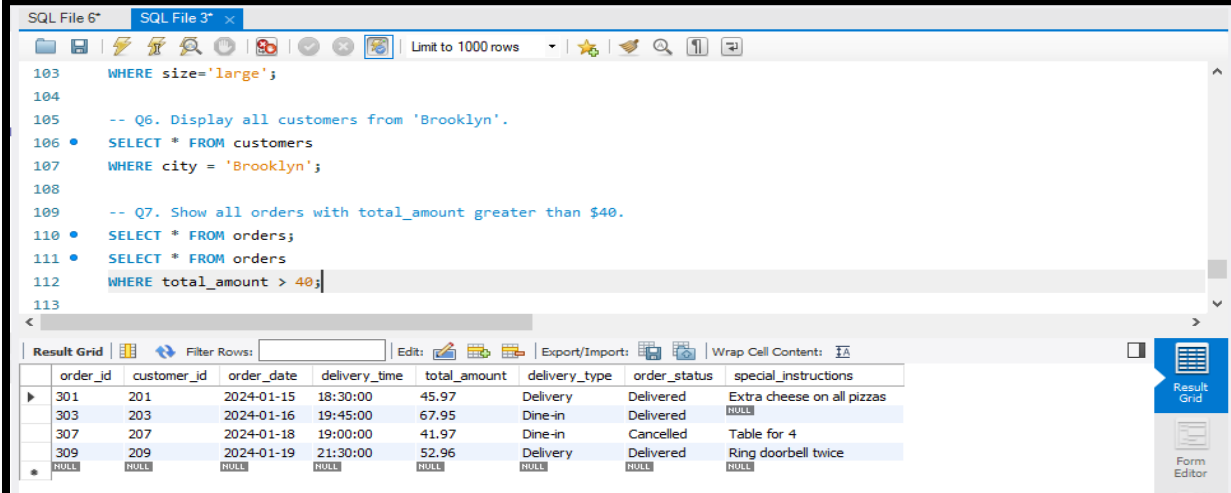
customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
202	Maria	Rossi	555-0102	maria.rossi@email.com	Brooklyn	2023-02-20	280	Regular
206	Sofia	Romano	555-0106	sofia.romano@email.com	Brooklyn	2023-04-12	65	Regular
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q7. Show all orders with total_amount greater than \$40.

```
SELECT * FROM orders;
```

```
SELECT * FROM orders
```

```
WHERE total_amount > 40;
```



The screenshot shows a SQL editor window with the following query:

```
103 WHERE size='large';
104
105 -- Q6. Display all customers from 'Brooklyn'.
106 • SELECT * FROM customers
107 WHERE city = 'Brooklyn';
108
109 -- Q7. Show all orders with total_amount greater than $40.
110 • SELECT * FROM orders;
111 • SELECT * FROM orders
112 WHERE total_amount > 40;
113
```

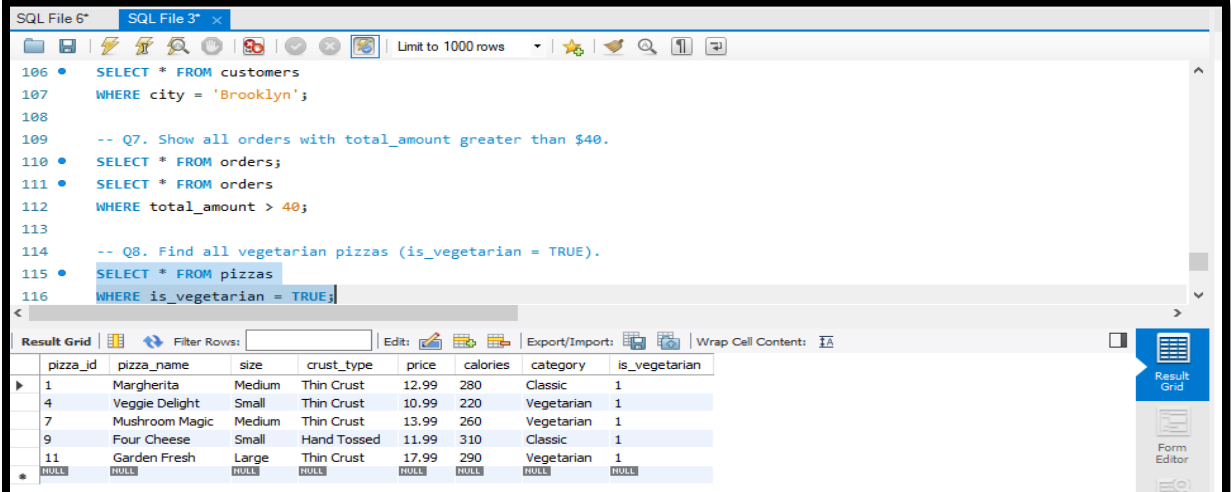
The results are displayed in a table grid with the following data:

order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
301	201	2024-01-15	18:30:00	45.97	Delivery	Delivered	Extra cheese on all pizzas
303	203	2024-01-16	19:45:00	67.95	Dine-in	Delivered	NULL
307	207	2024-01-18	19:00:00	41.97	Dine-in	Cancelled	Table for 4
309	209	2024-01-19	21:30:00	52.96	Delivery	Delivered	Ring doorbell twice
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q8. Find all vegetarian pizzas (is_vegetarian = TRUE).

```
SELECT * FROM pizzas
```

```
WHERE is_vegetarian = TRUE;
```



The screenshot shows a SQL editor window with the following query:

```
106 • SELECT * FROM customers
107 WHERE city = 'Brooklyn';
108
109 -- Q7. Show all orders with total_amount greater than $40.
110 • SELECT * FROM orders;
111 • SELECT * FROM orders
112 WHERE total_amount > 40;
113
114 -- Q8. Find all vegetarian pizzas (is_vegetarian = TRUE).
115 • SELECT * FROM pizzas
116 WHERE is_vegetarian = TRUE;
```

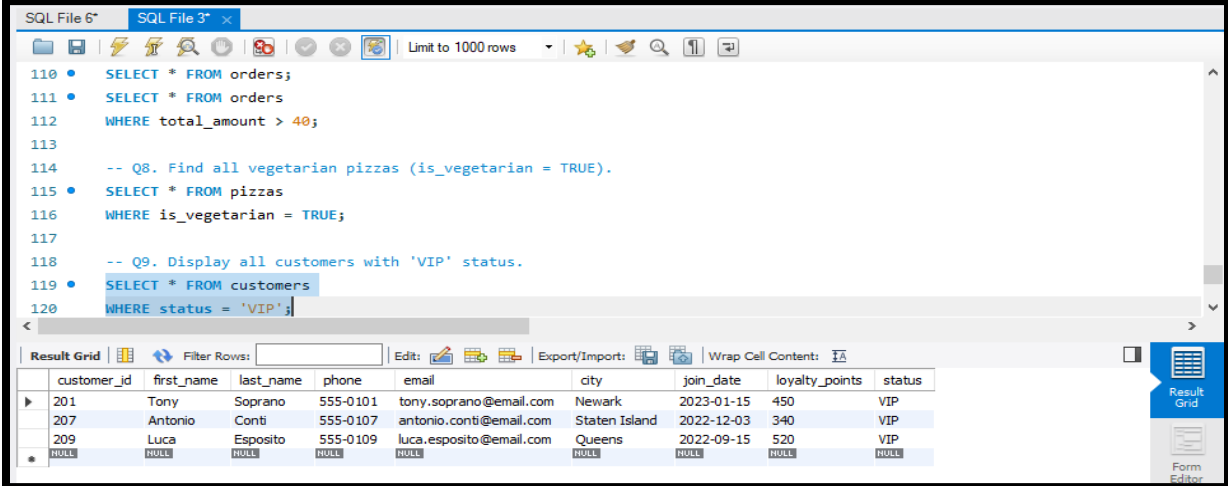
The results are displayed in a table grid with the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
4	Veggie Delight	Small	Thin Crust	10.99	220	Vegetarian	1
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
9	Four Cheese	Small	Hand Tossed	11.99	310	Classic	1
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q9. Display all customers with 'VIP' status.

```
SELECT * FROM customers
```

```
WHERE status = 'VIP';
```



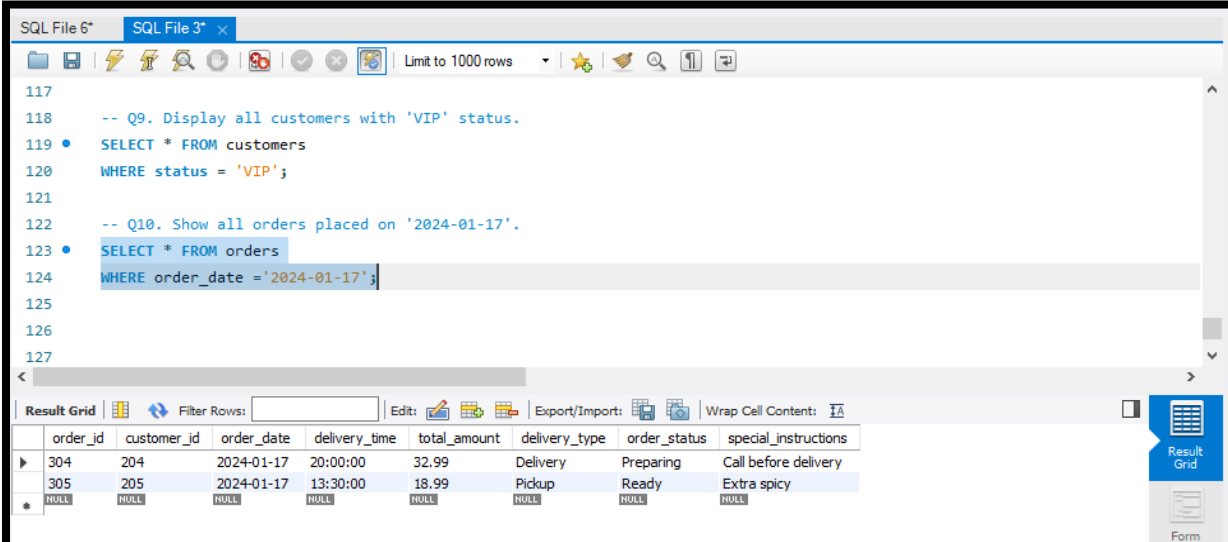
The screenshot shows a SQL editor window with two tabs: 'SQL File 6*' and 'SQL File 3*'. The 'SQL File 3*' tab is active, displaying a series of SQL queries. The query for Q9 is highlighted: `SELECT * FROM customers WHERE status = 'VIP';`. Below the editor, a 'Result Grid' is visible, showing the results of the query. The grid has columns for customer_id, first_name, last_name, phone, email, city, join_date, loyalty_points, and status. The results show three rows of data for VIP customers.

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP
209	Luca	Esposito	555-0109	luca.esposito@email.com	Queens	2022-09-15	520	VIP
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Show all orders placed on '2024-01-17'.

```
SELECT * FROM orders
```

```
WHERE order_date = '2024-01-17';
```



The screenshot shows a SQL editor window with two tabs: 'SQL File 6*' and 'SQL File 3*'. The 'SQL File 3*' tab is active, displaying a series of SQL queries. The query for Q10 is highlighted: `SELECT * FROM orders WHERE order_date = '2024-01-17';`. Below the editor, a 'Result Grid' is visible, showing the results of the query. The grid has columns for order_id, customer_id, order_date, delivery_time, total_amount, delivery_type, order_status, and special_instructions. The results show two rows of data for orders placed on 2024-01-17.

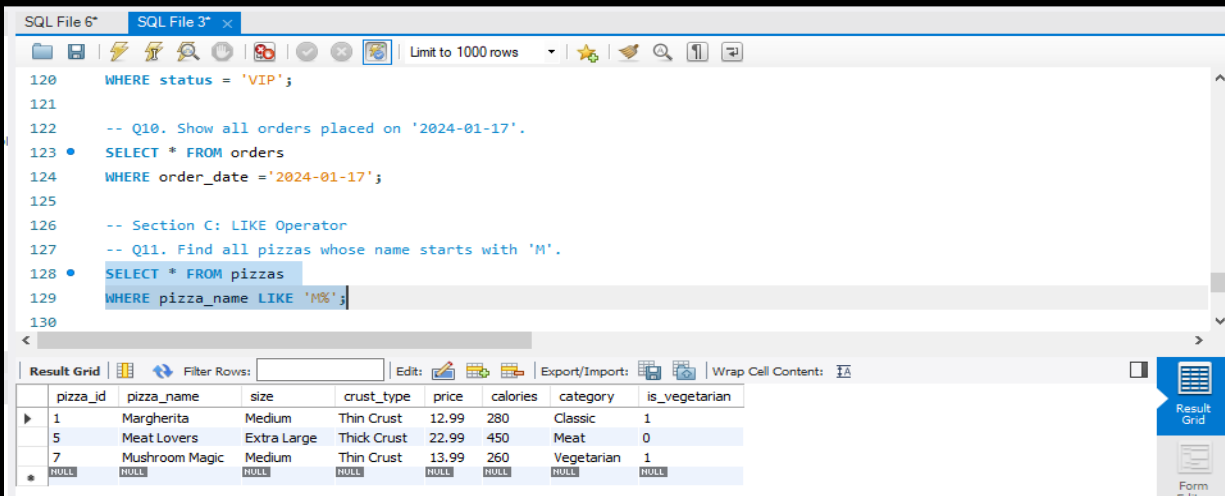
order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
304	204	2024-01-17	20:00:00	32.99	Delivery	Preparing	Call before delivery
305	205	2024-01-17	13:30:00	18.99	Pickup	Ready	Extra spicy
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section C: LIKE Operator

Q11. Find all pizzas whose name starts with 'M'.

```
SELECT * FROM pizzas
```

```
WHERE pizza_name LIKE 'M%';
```



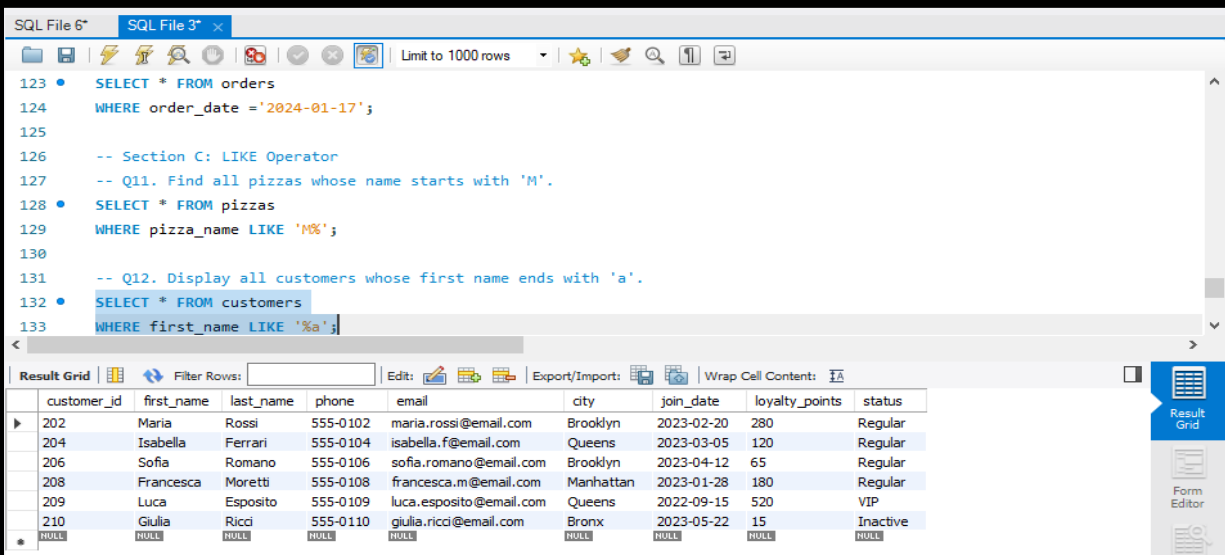
```
120 WHERE status = 'VIP';
121
122 -- Q10. Show all orders placed on '2024-01-17'.
123 • SELECT * FROM orders
124 WHERE order_date = '2024-01-17';
125
126 -- Section C: LIKE Operator
127 -- Q11. Find all pizzas whose name starts with 'M'.
128 • SELECT * FROM pizzas
129 WHERE pizza_name LIKE 'M%';
130
```

	pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
▶	1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
	5	Meat Lovers	Extra Large	Thick Crust	22.99	450	Meat	0
	7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q12. Display all customers whose first name ends with 'a'.

```
SELECT * FROM customers
```

```
WHERE first_name LIKE '%a';
```



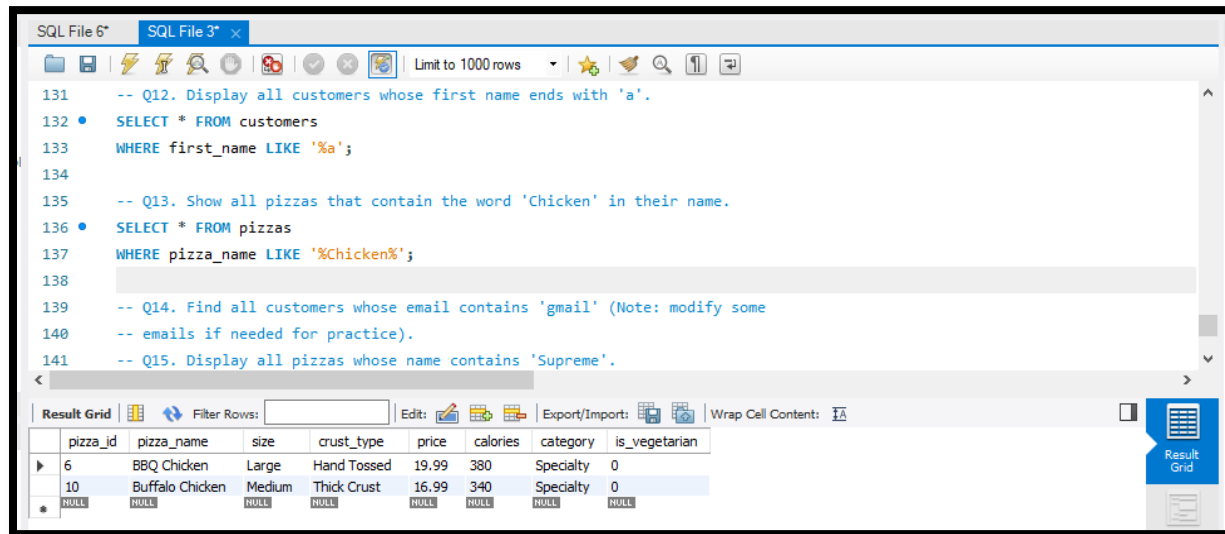
```
123 • SELECT * FROM orders
124 WHERE order_date = '2024-01-17';
125
126 -- Section C: LIKE Operator
127 -- Q11. Find all pizzas whose name starts with 'M'.
128 • SELECT * FROM pizzas
129 WHERE pizza_name LIKE 'M%';
130
131 -- Q12. Display all customers whose first name ends with 'a'.
132 • SELECT * FROM customers
133 WHERE first_name LIKE '%a';
```

	customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
▶	202	Maria	Rossi	555-0102	maria.rossi@email.com	Brooklyn	2023-02-20	280	Regular
	204	Isabella	Ferrari	555-0104	isabella.f@email.com	Queens	2023-03-05	120	Regular
	206	Sofia	Romano	555-0106	sofia.romano@email.com	Brooklyn	2023-04-12	65	Regular
	208	Francesca	Moretti	555-0108	francesca.m@email.com	Manhattan	2023-01-28	180	Regular
	209	Luca	Esposito	555-0109	luca.esposito@email.com	Queens	2022-09-15	520	VIP
	210	Giulia	Ricci	555-0110	giulia.ricci@email.com	Bronx	2023-05-22	15	Inactive
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q13. Show all pizzas that contain the word 'Chicken' in their name.

```
SELECT * FROM pizzas
```

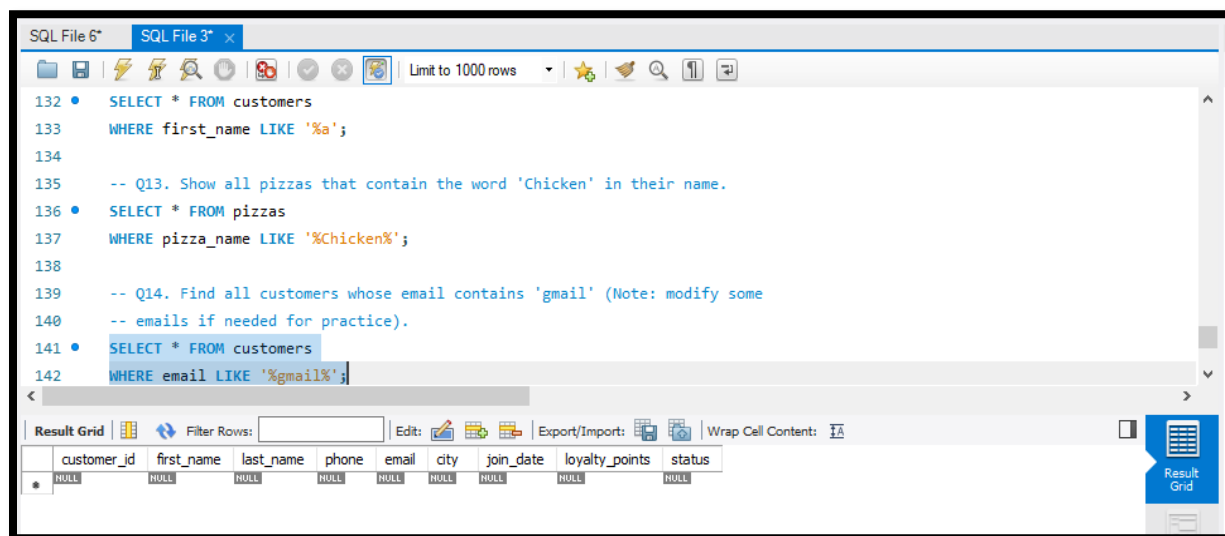
```
WHERE pizza_name LIKE '%Chicken%';
```



Q14. Find all customers whose email contains 'gmail' (Note: modify some emails if needed for practice).

```
SELECT * FROM customers
```

```
WHERE email LIKE '%gmail%';
```



Modify some emails if needed for practice

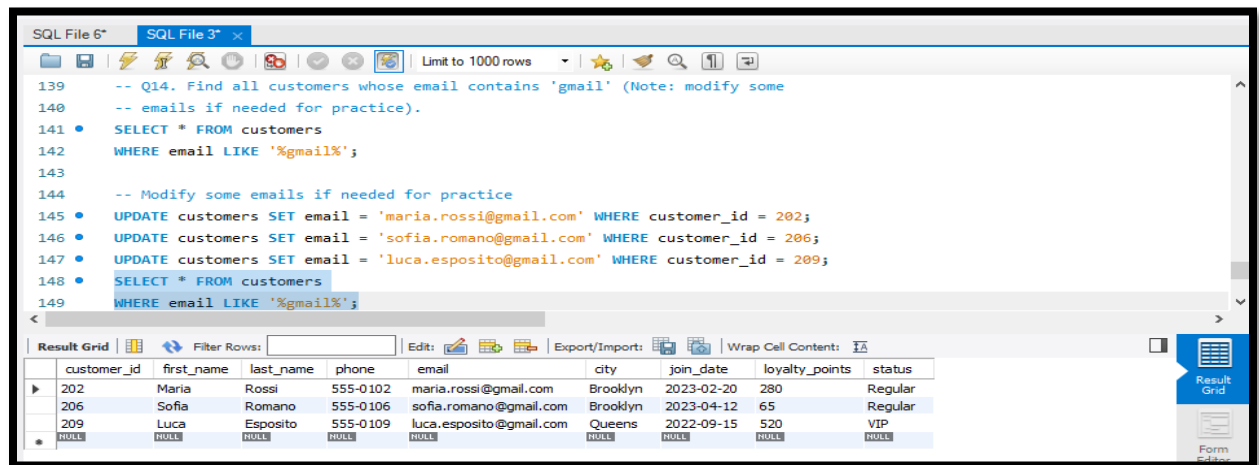
UPDATE customers SET email = 'maria.rossi@gmail.com' WHERE customer_id = 202;

UPDATE customers SET email = 'sofia.romano@gmail.com' WHERE customer_id = 206;

UPDATE customers SET email = 'luca.esposito@gmail.com' WHERE customer_id = 209;

SELECT * FROM customers

WHERE email LIKE '%gmail%';



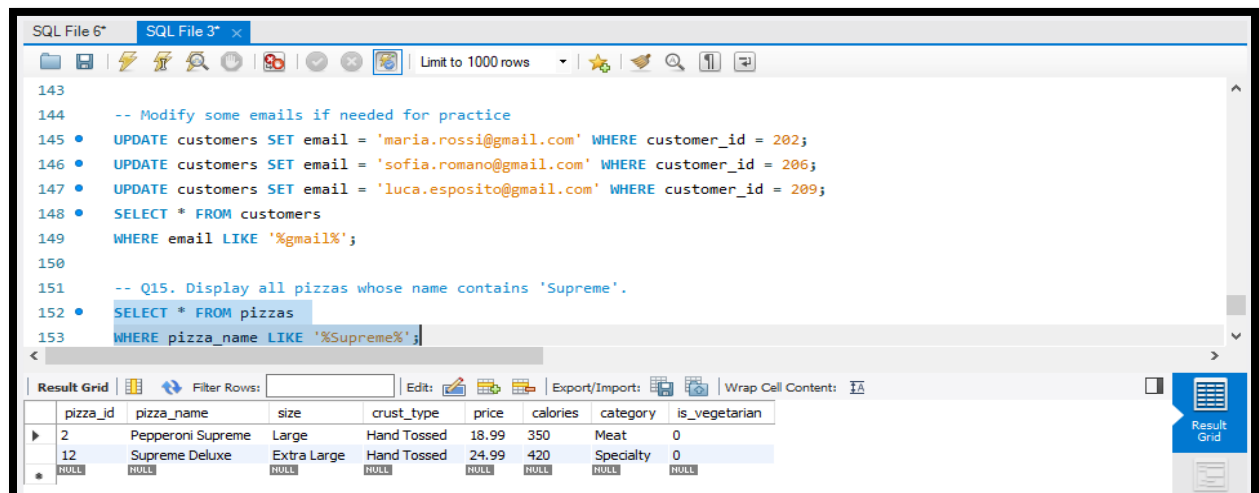
The screenshot shows a SQL editor window titled 'SQL File 3*'. The SQL code includes comments for Q14 and Q15, and several UPDATE and SELECT statements. The result grid below shows the output of the SELECT statement for Q14.

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
202	Maria	Rossi	555-0102	maria.rossi@gmail.com	Brooklyn	2023-02-20	280	Regular
206	Sofia	Romano	555-0106	sofia.romano@gmail.com	Brooklyn	2023-04-12	65	Regular
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q15. Display all pizzas whose name contains 'Supreme'.

SELECT * FROM pizzas

WHERE pizza_name LIKE '%Supreme%';



The screenshot shows a SQL editor window titled 'SQL File 3*'. The SQL code includes comments for Q15 and a SELECT statement. The result grid below shows the output of the SELECT statement for Q15.

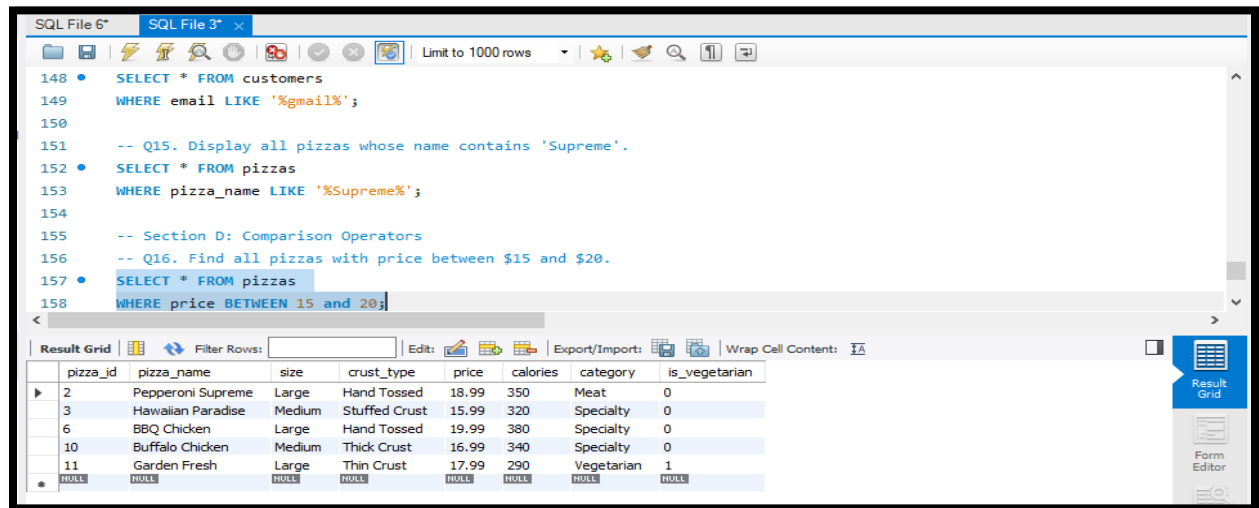
pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
12	Supreme Deluxe	Extra Large	Hand Tossed	24.99	420	Specialty	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section D: Comparison Operators

Q16. Find all pizzas with price between \$15 and \$20.

```
SELECT * FROM pizzas
```

```
WHERE price BETWEEN 15 and 20;
```



The screenshot shows a SQL editor window with the following SQL code:

```
148 • SELECT * FROM customers
149 WHERE email LIKE '%gmail%';
150
151 -- Q15. Display all pizzas whose name contains 'Supreme'.
152 • SELECT * FROM pizzas
153 WHERE pizza_name LIKE '%Supreme%';
154
155 -- Section D: Comparison Operators
156 -- Q16. Find all pizzas with price between $15 and $20.
157 • SELECT * FROM pizzas
158 WHERE price BETWEEN 15 and 20;
```

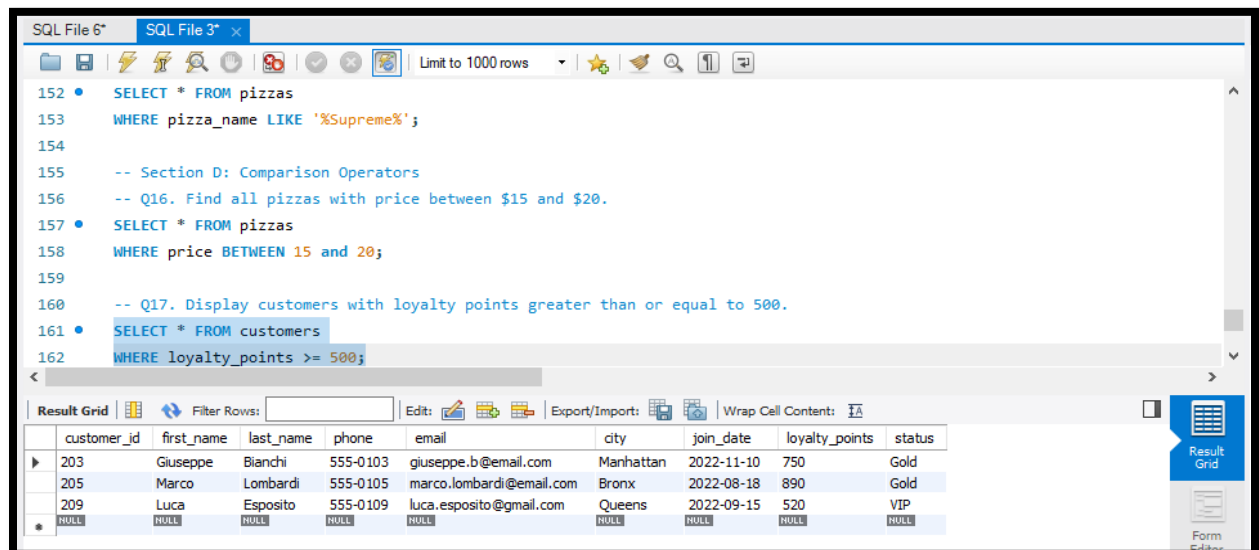
The result grid displays the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
3	Hawaiian Paradise	Medium	Stuffed Crust	15.99	320	Specialty	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q17. Display customers with loyalty points greater than or equal to 500.

```
SELECT * FROM customers
```

```
WHERE loyalty_points >= 500;
```



The screenshot shows a SQL editor window with the following SQL code:

```
152 • SELECT * FROM pizzas
153 WHERE pizza_name LIKE '%Supreme%';
154
155 -- Section D: Comparison Operators
156 -- Q16. Find all pizzas with price between $15 and $20.
157 • SELECT * FROM pizzas
158 WHERE price BETWEEN 15 and 20;
159
160 -- Q17. Display customers with loyalty points greater than or equal to 500.
161 • SELECT * FROM customers
162 WHERE loyalty_points >= 500;
```

The result grid displays the following data:

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
203	Giuseppe	Bianchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
209	Luca	Eposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q18. Show all pizzas with calories less than 300.

```
SELECT * FROM pizzas
```

```
WHERE calories < 300;
```

The screenshot shows a SQL editor window titled 'SQL File 4' with the following SQL code:

```
-- Q17. Display customers with loyalty points greater than or equal to 500.
159 • SELECT * FROM customers
160 WHERE loyalty_points >= 500;
161
162
163 -- Q18. Show all pizzas with calories less than 300.
164 • SELECT * FROM pizzas
165 WHERE calories < 300;
166
167 -- Q19. Find orders with total_amount less than or equal to $25.
168 • SELECT * FROM orders
```

The result grid below the code shows the results of the query for Q18:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
4	Veggie Delight	Small	Thin Crust	10.99	220	Vegetarian	1
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q19. Find orders with total_amount less than or equal to \$25.

```
SELECT * FROM orders
```

```
WHERE total_amount <= 25;
```

The screenshot shows a SQL editor window titled 'SQL File 3' with the following SQL code:

```
-- Q17. Display customers with loyalty points greater than or equal to 500.
160 • SELECT * FROM customers
161 WHERE loyalty_points >= 500;
162
163
164 -- Q18. Show all pizzas with calories less than 300.
165 • SELECT * FROM pizzas
166 WHERE calories < 300;
167
168 -- Q19. Find orders with total_amount less than or equal to $25.
169 • SELECT * FROM orders
170 WHERE total_amount <= 25;
```

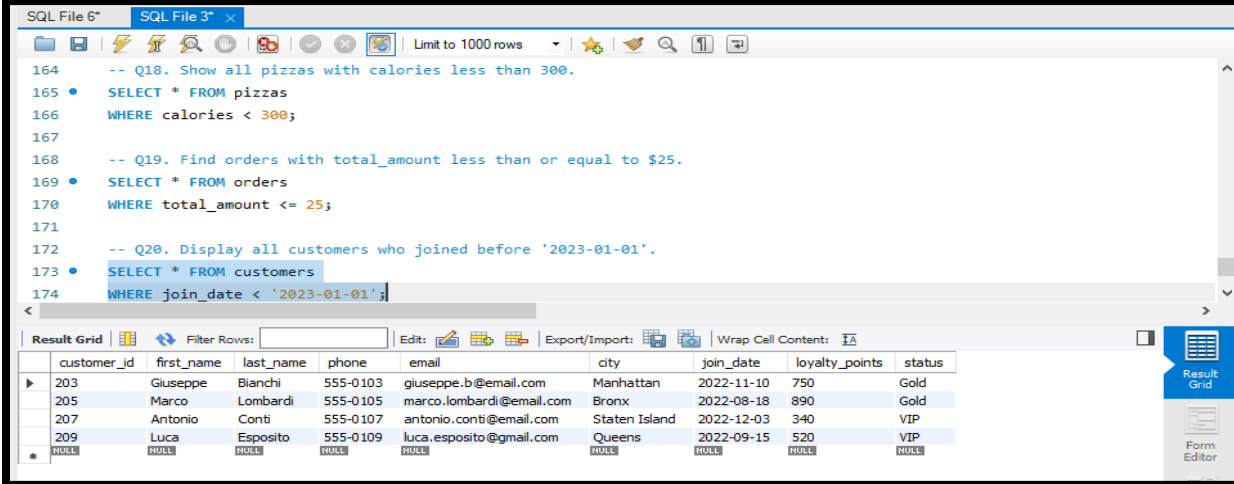
The result grid below the code shows the results of the query for Q19:

order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
302	202	2024-01-16	12:15:00	23.98	Pickup	Ready	No onions please
305	205	2024-01-17	13:30:00	18.99	Pickup	Ready	Extra spicy
308	208	2024-01-19	14:45:00	15.99	Pickup	Delivered	Light sauce
310	210	2024-01-20	16:20:00	12.99	Pickup	Pending	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q20. Display all customers who joined before '2023-01-01'.

```
SELECT * FROM customers
```

```
WHERE join_date < '2023-01-01';
```



The screenshot shows a SQL editor window with the following code:

```
164 -- Q18. Show all pizzas with calories less than 300.
165 • SELECT * FROM pizzas
166 WHERE calories < 300;
167
168 -- Q19. Find orders with total_amount less than or equal to $25.
169 • SELECT * FROM orders
170 WHERE total_amount <= 25;
171
172 -- Q20. Display all customers who joined before '2023-01-01'.
173 • SELECT * FROM customers
174 WHERE join_date < '2023-01-01';
```

The results are displayed in a table grid with the following data:

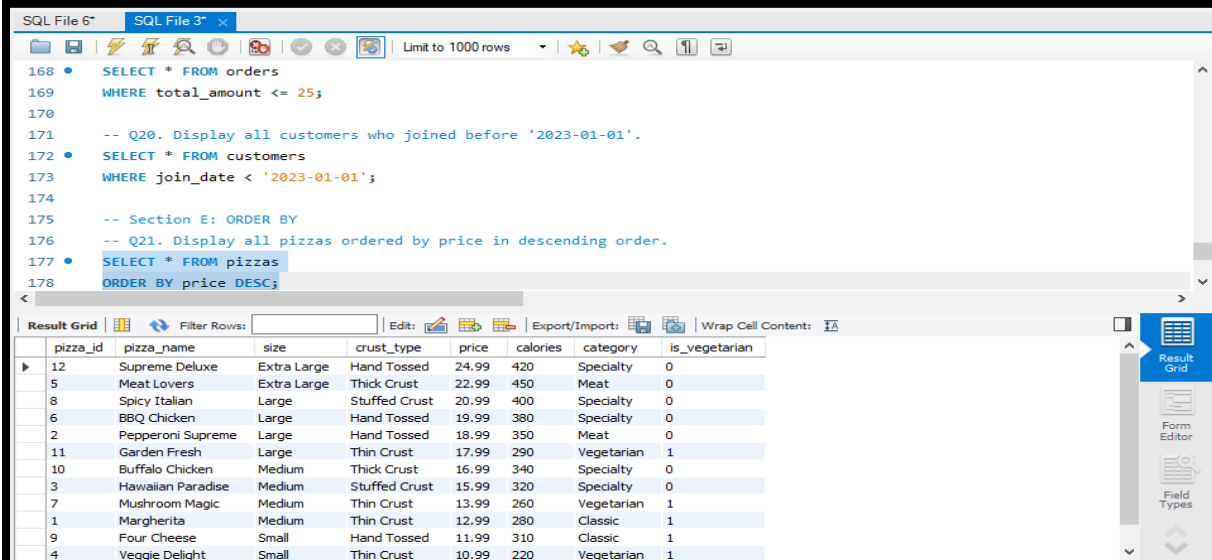
customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
203	Giuseppe	Bianchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section E: ORDER BY

Q21. Display all pizzas ordered by price in descending order.

```
SELECT * FROM pizzas
```

```
ORDER BY price DESC;
```



The screenshot shows a SQL editor window with the following code:

```
168 • SELECT * FROM orders
169 WHERE total_amount <= 25;
170
171 -- Q20. Display all customers who joined before '2023-01-01'.
172 • SELECT * FROM customers
173 WHERE join_date < '2023-01-01';
174
175 -- Section E: ORDER BY
176 -- Q21. Display all pizzas ordered by price in descending order.
177 • SELECT * FROM pizzas
178 ORDER BY price DESC;
```

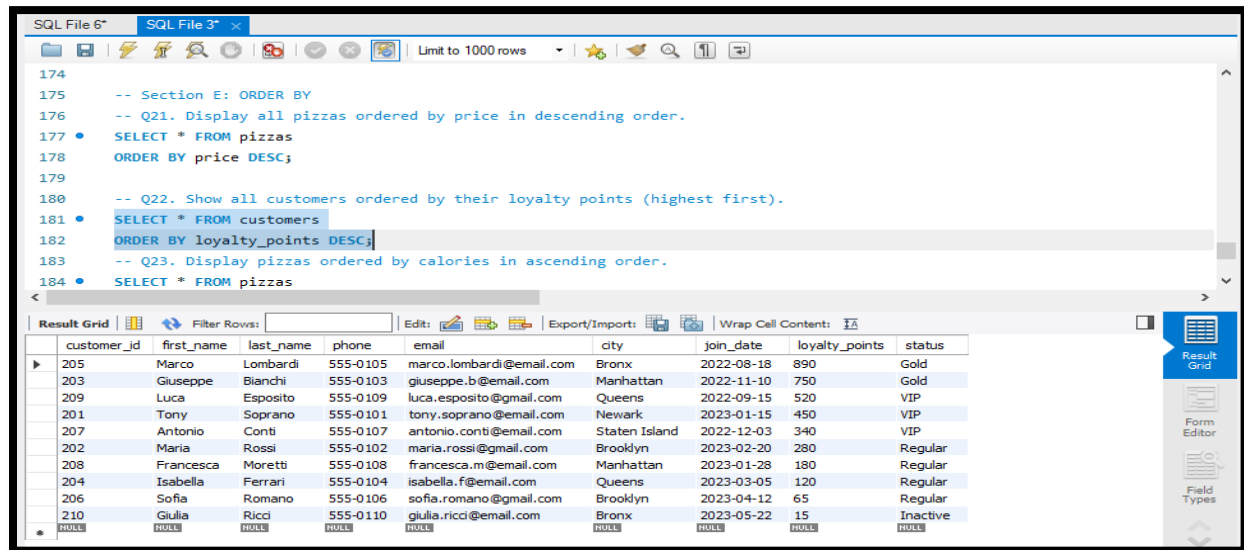
The results are displayed in a table grid with the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
12	Supreme Deluxe	Extra Large	Hand Tossed	24.99	420	Specialty	0
5	MeatLovers	Extra Large	Thick Crust	22.99	450	Meat	0
8	Spicy Italian	Large	Stuffed Crust	20.99	400	Specialty	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
3	Hawaiian Paradise	Medium	Stuffed Crust	15.99	320	Specialty	0
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
9	Four Cheese	Small	Hand Tossed	11.99	310	Classic	1
4	Veggie Delight	Small	Thin Crust	10.99	220	Vegetarian	1

Q22. Show all customers ordered by their loyalty points (highest first).

```
SELECT * FROM customers
```

```
ORDER BY loyalty_points DESC;
```



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
174
175 -- Section E: ORDER BY
176 -- Q21. Display all pizzas ordered by price in descending order.
177 • SELECT * FROM pizzas
178   ORDER BY price DESC;
179
180 -- Q22. Show all customers ordered by their loyalty points (highest first).
181 • SELECT * FROM customers
182   ORDER BY loyalty_points DESC;
183 -- Q23. Display pizzas ordered by calories in ascending order.
184 • SELECT * FROM pizzas
```

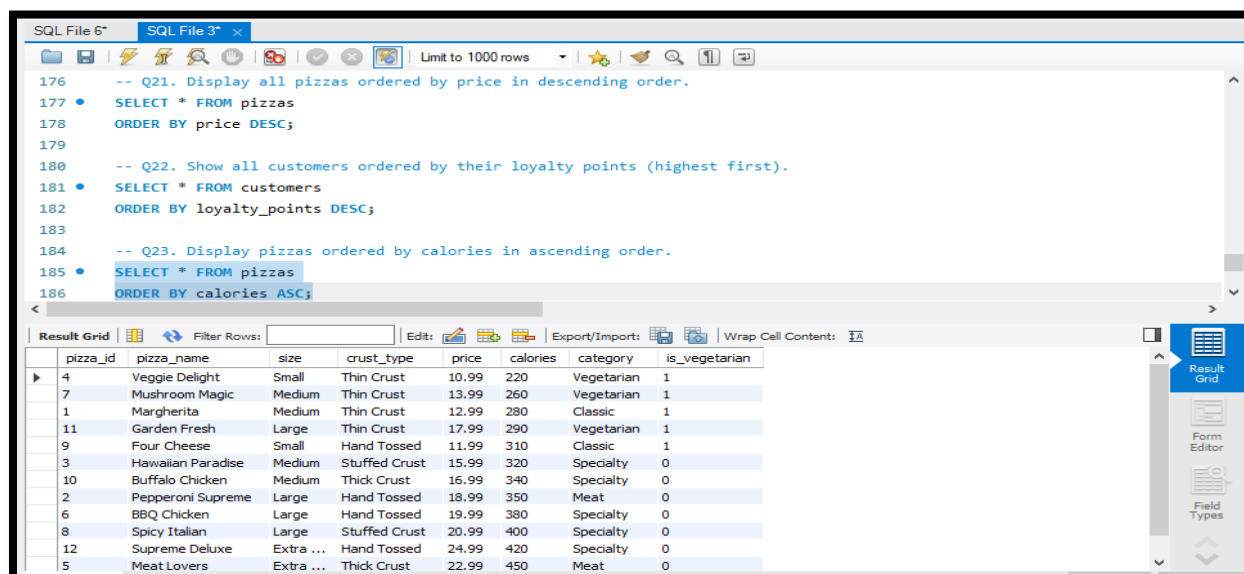
The result grid displays the following data:

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
203	Giuseppe	Bianchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
209	Luca	Espósito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP
202	Maria	Rossi	555-0102	maria.rossi@gmail.com	Brooklyn	2023-02-20	280	Regular
208	Francesca	Moretti	555-0108	francesca.m@email.com	Manhattan	2023-01-28	180	Regular
204	Isabella	Ferrari	555-0104	isabella.f@email.com	Queens	2023-03-05	120	Regular
206	Sofia	Romano	555-0106	sofia.romano@gmail.com	Brooklyn	2023-04-12	65	Regular
210	Giulia	Ricci	555-0110	giulia.ricci@email.com	Bronx	2023-05-22	15	Inactive

Q23. Display pizzas ordered by calories in ascending order.

```
SELECT * FROM pizzas
```

```
ORDER BY calories ASC;
```



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
176 -- Q21. Display all pizzas ordered by price in descending order.
177 • SELECT * FROM pizzas
178   ORDER BY price DESC;
179
180 -- Q22. Show all customers ordered by their loyalty points (highest first).
181 • SELECT * FROM customers
182   ORDER BY loyalty_points DESC;
183
184 -- Q23. Display pizzas ordered by calories in ascending order.
185 • SELECT * FROM pizzas
186   ORDER BY calories ASC;
```

The result grid displays the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
4	Veggie Delight	Small	Thin Crust	10.99	220	Vegetarian	1
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
1	Margherita	Medium	Thin Crust	12.99	280	Classic	1
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
9	Four Cheese	Small	Hand Tossed	11.99	310	Classic	1
3	Hawaiian Paradise	Medium	Stuffed Crust	15.99	320	Specialty	0
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0
8	Spicy Italian	Large	Stuffed Crust	20.99	400	Specialty	0
12	Supreme Deluxe	Extra ...	Hand Tossed	24.99	420	Specialty	0
5	Meat Lovers	Extra ...	Thick Crust	22.99	450	Meat	0

Q24. Show customers ordered by city name alphabetically, then by last name.

```
SELECT * FROM customers
```

```
ORDER BY city ASC, last_name ASC;
```

SQL File 3

```
-- Q22. Show all customers ordered by their loyalty points (highest first).
180
181 • SELECT * FROM customers
182 ORDER BY loyalty_points DESC;
183
-- Q23. Display pizzas ordered by calories in ascending order.
184
185 • SELECT * FROM pizzas
186 ORDER BY calories ASC;
187
-- Q24. Show customers ordered by city name alphabetically, then by last name.
188
189 • SELECT * FROM customers
190 ORDER BY city ASC, last_name ASC;
```

Result Grid

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
210	Giulia	Ricci	555-0110	giulia.ricci@email.com	Bronx	2023-05-22	15	Inactive
206	Sofia	Romano	555-0106	sofia.romano@gmail.com	Brooklyn	2023-04-12	65	Regular
202	Maria	Rossi	555-0102	maria.rossi@gmail.com	Brooklyn	2023-02-20	280	Regular
203	Giuseppe	Bianchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
208	Francesca	Moretti	555-0108	francesca.m@email.com	Manhattan	2023-01-28	180	Regular
201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
204	Isabella	Ferrari	555-0104	isabella.f@email.com	Queens	2022-03-05	120	Regular
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP

Q25. Display orders ordered by order_date (newest first), then by total_amount (highest first).

```
SELECT * FROM orders
```

```
ORDER BY order_date DESC, total_amount DESC;
```

SQL File 3

```
-- Q23. Display pizzas ordered by calories in ascending order.
184
185 • SELECT * FROM pizzas
186 ORDER BY calories ASC;
187
-- Q24. Show customers ordered by city name alphabetically, then by last name.
188
189 • SELECT * FROM customers
190 ORDER BY city ASC, last_name ASC;
191
-- Q25. Display orders ordered by order_date (newest first), then by total_amount (highest first).
192
193 • SELECT * FROM orders
194 ORDER BY order_date DESC, total_amount DESC;
```

Result Grid

order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
310	210	2024-01-20	16:20:00	12.99	Pickup	Pending	
309	209	2024-01-19	21:30:00	52.96	Delivery	Delivered	Ring doorbell twice
308	208	2024-01-19	14:45:00	15.99	Pickup	Delivered	Light sauce
307	207	2024-01-18	19:00:00	41.97	Dine-in	Cancelled	Table for 4
306	206	2024-01-18	17:15:00	28.98	Delivery	Delivered	
304	204	2024-01-17	20:00:00	32.99	Delivery	Preparing	Call before delivery
305	205	2024-01-17	13:30:00	18.99	Pickup	Ready	Extra spicy
303	203	2024-01-16	19:45:00	67.95	Dine-in	Delivered	
302	202	2024-01-16	12:15:00	23.98	Pickup	Ready	No onions please
301	201	2024-01-15	18:30:00	45.97	Delivery	Delivered	Extra cheese on all pizzas

Section F: LIMIT and OFFSET

Q26. Show the top 3 most expensive pizzas.

```
SELECT * FROM pizzas
```

```
ORDER BY price DESC
```

```
limit 3;
```

The screenshot shows a SQL editor window with the following code:

```
192 -- Q25. Display orders ordered by order_date (newest first), then by total_amount (highest first).
193 • SELECT * FROM orders
194 ORDER BY order_date DESC, total_amount DESC;
195
196 -- Section F: LIMIT and OFFSET
197 -- Q26. Show the top 3 most expensive pizzas.
198 • SELECT * FROM pizzas
199 ORDER BY price DESC
200 limit 3;
201
202 -- Q27. Display the 5 customers with the most loyalty points.
```

The Result Grid shows the top 3 most expensive pizzas:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
12	Supreme Deluxe	Extra Large	Hand Tossed	24.99	420	Specialty	0
5	Meat Lovers	Extra Large	Thick Crust	22.99	450	Meat	0
8	Spicy Italian	Large	Stuffed Crust	20.99	400	Specialty	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q27. Display the 5 customers with the most loyalty points.

```
SELECT * FROM customers
```

```
ORDER BY loyalty_points DESC
```

```
limit 5;
```

The screenshot shows a SQL editor window with the following code:

```
196 -- Section F: LIMIT and OFFSET
197 -- Q26. Show the top 3 most expensive pizzas.
198 • SELECT * FROM pizzas
199 ORDER BY price DESC
200 limit 3;
201
202 -- Q27. Display the 5 customers with the most loyalty points.
203 • SELECT * FROM customers
204 ORDER BY loyalty_points DESC
205 limit 5;
206 -- Q28. Show pizzas ranked 4-6 when ordered by price (use OFFSET).
```

The Result Grid shows the top 5 customers with the most loyalty points:

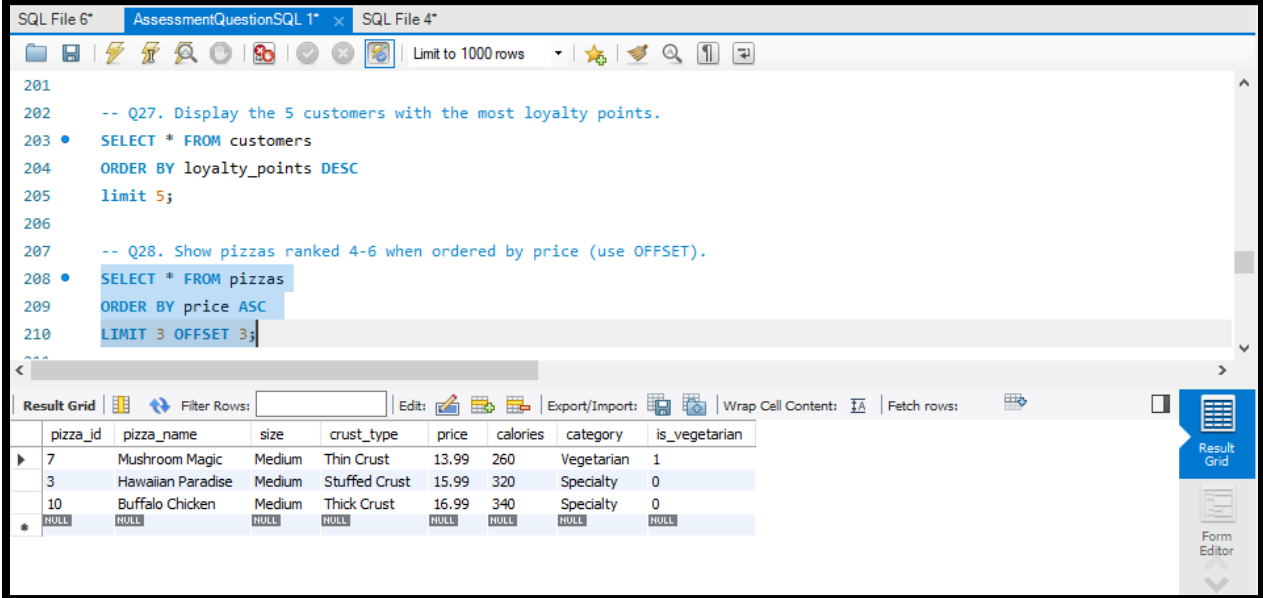
customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
205	Marco	Lombardi	555-0105	marco.lombardi@email.com	Bronx	2022-08-18	890	Gold
203	Giuseppe	Blanchi	555-0103	giuseppe.b@email.com	Manhattan	2022-11-10	750	Gold
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q28. Show pizzas ranked 4-6 when ordered by price (use OFFSET).

```
SELECT * FROM pizzas
```

```
ORDER BY price ASC
```

```
LIMIT 3 OFFSET 3;
```



SQL File 6* AssessmentQuestionSQL 1* SQL File 4*

Limit to 1000 rows

```
201
202 -- Q27. Display the 5 customers with the most loyalty points.
203 • SELECT * FROM customers
204 ORDER BY loyalty_points DESC
205 limit 5;
206
207 -- Q28. Show pizzas ranked 4-6 when ordered by price (use OFFSET).
208 • SELECT * FROM pizzas
209 ORDER BY price ASC
210 LIMIT 3 OFFSET 3;
```

Result Grid

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
7	Mushroom Magic	Medium	Thin Crust	13.99	260	Vegetarian	1
3	Hawaiian Paradise	Medium	Stuffed Crust	15.99	320	Specialty	0
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

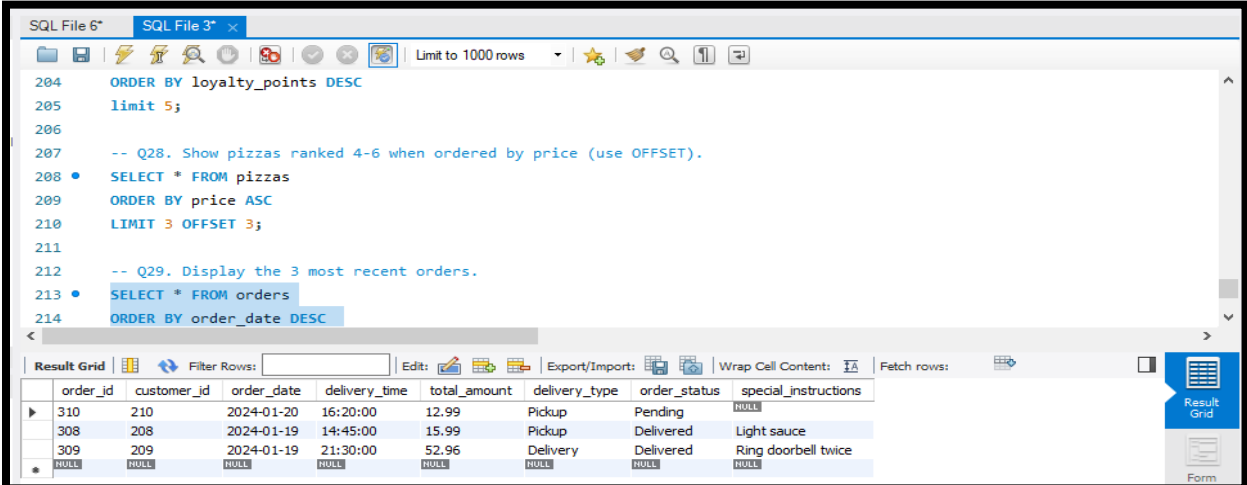
Form Editor

Q29. Display the 3 most recent orders.

```
SELECT * FROM orders
```

```
ORDER BY order_date DESC
```

```
LIMIT 3;
```



SQL File 6* SQL File 3*

Limit to 1000 rows

```
204 ORDER BY loyalty_points DESC
205 limit 5;
206
207 -- Q28. Show pizzas ranked 4-6 when ordered by price (use OFFSET).
208 • SELECT * FROM pizzas
209 ORDER BY price ASC
210 LIMIT 3 OFFSET 3;
211
212 -- Q29. Display the 3 most recent orders.
213 • SELECT * FROM orders
214 ORDER BY order_date DESC
```

Result Grid

order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
310	210	2024-01-20	16:20:00	12.99	Pickup	Pending	NULL
308	208	2024-01-19	14:45:00	15.99	Pickup	Delivered	Light sauce
309	209	2024-01-19	21:30:00	52.96	Delivery	Delivered	Ring doorbell twice
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

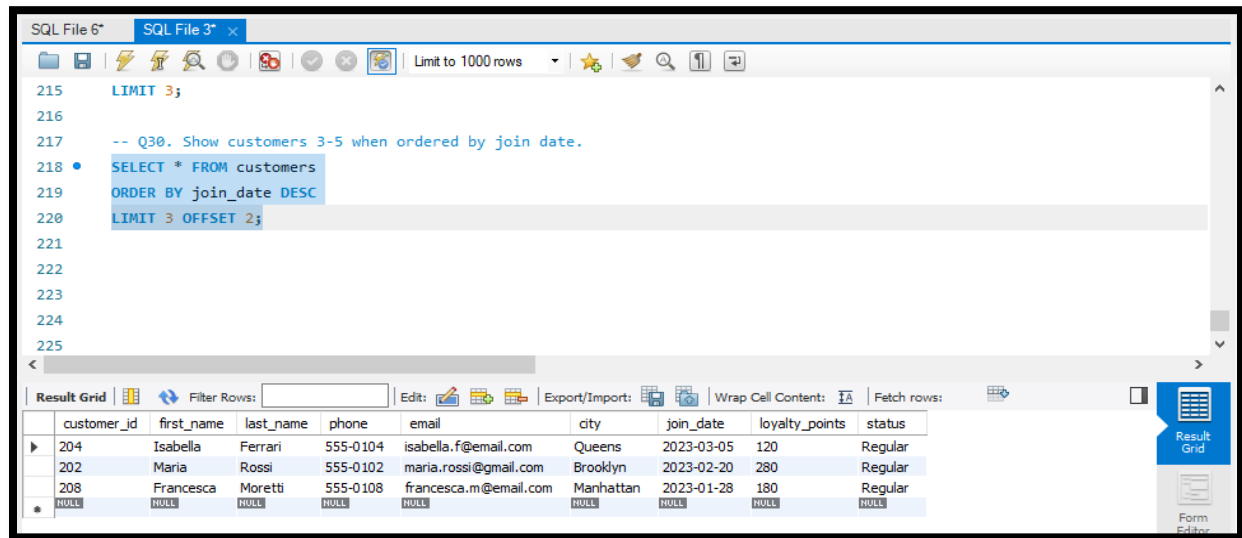
Form

Q30. Show customers 3-5 when ordered by join date.

```
SELECT * FROM customers
```

```
ORDER BY join_date DESC
```

```
LIMIT 3 OFFSET 2;
```



The screenshot shows a SQL IDE with a query editor and a result grid. The query editor contains the following SQL code:

```
215 LIMIT 3;  
216  
217 -- Q30. Show customers 3-5 when ordered by join date.  
218 SELECT * FROM customers  
219 ORDER BY join_date DESC  
220 LIMIT 3 OFFSET 2;  
221  
222  
223  
224  
225
```

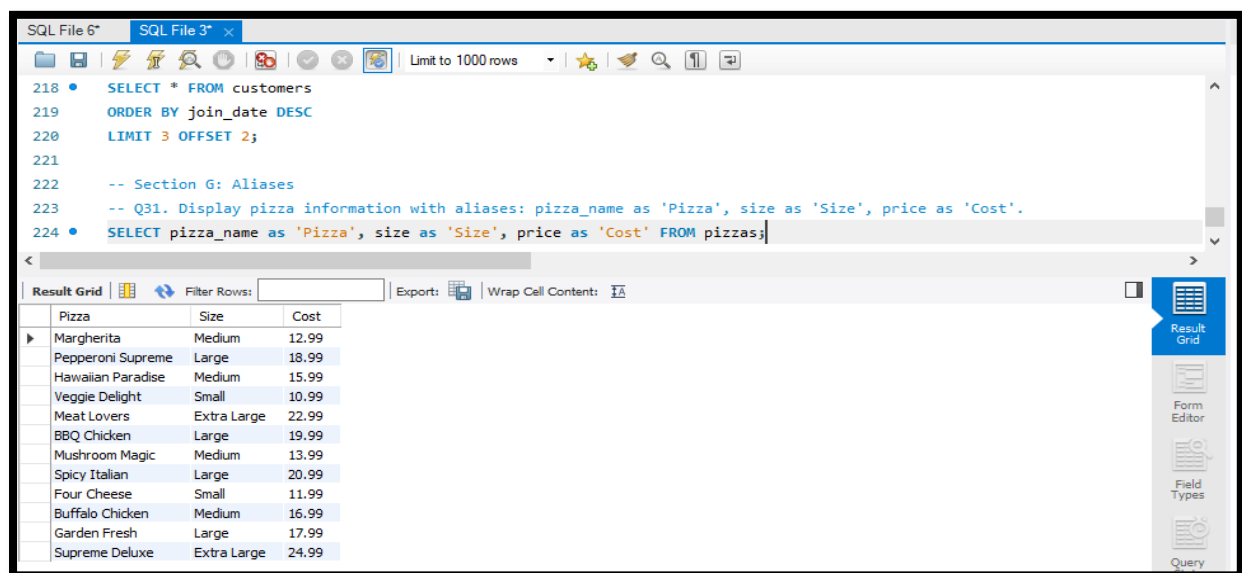
The result grid displays the following data:

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
204	Isabella	Ferrari	555-0104	isabella.f@email.com	Queens	2023-03-05	120	Regular
202	Maria	Rossi	555-0102	maria.rossi@gmail.com	Brooklyn	2023-02-20	280	Regular
208	Francesca	Moretti	555-0108	francesca.m@email.com	Manhattan	2023-01-28	180	Regular
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section G: Aliases

Q31. Display pizza information with aliases: pizza_name as 'Pizza', size as 'Size', price as 'Cost'.

```
SELECT pizza_name as 'Pizza', size as 'Size', price as 'Cost' FROM pizzas;
```



The screenshot shows a SQL IDE with a query editor and a result grid. The query editor contains the following SQL code:

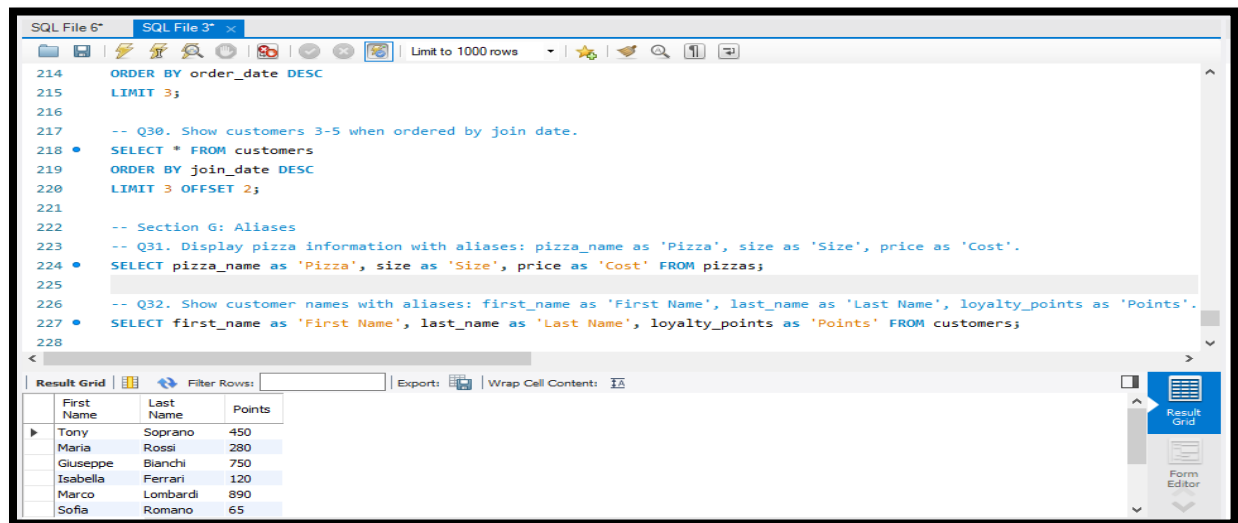
```
218 SELECT * FROM customers  
219 ORDER BY join_date DESC  
220 LIMIT 3 OFFSET 2;  
221  
222 -- Section G: Aliases  
223 -- Q31. Display pizza information with aliases: pizza_name as 'Pizza', size as 'Size', price as 'Cost'.  
224 SELECT pizza_name as 'Pizza', size as 'Size', price as 'Cost' FROM pizzas;
```

The result grid displays the following data:

Pizza	Size	Cost
Margherita	Medium	12.99
Pepperoni Supreme	Large	18.99
Hawaiian Paradise	Medium	15.99
Veggie Delight	Small	10.99
Meat Lovers	Extra Large	22.99
BBQ Chicken	Large	19.99
Mushroom Magic	Medium	13.99
Spicy Italian	Large	20.99
Four Cheese	Small	11.99
Buffalo Chicken	Medium	16.99
Garden Fresh	Large	17.99
Supreme Deluxe	Extra Large	24.99

Q32. Show customer names with aliases: first_name as 'First Name', last_name as 'Last Name', loyalty_points as 'Points'.

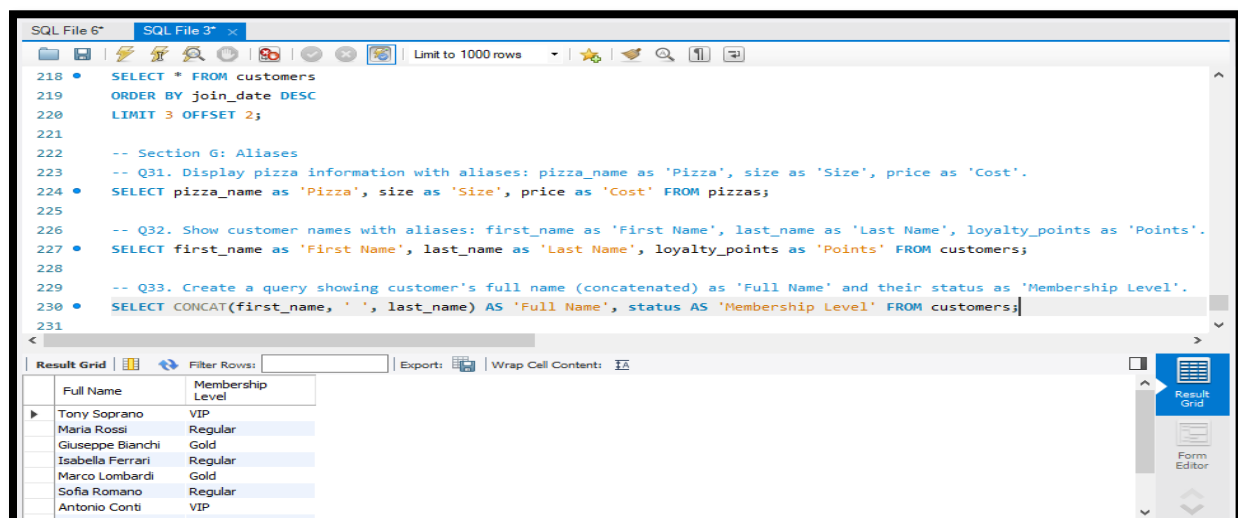
SELECT first_name as 'First Name', last_name as 'Last Name', loyalty_points as 'Points' FROM customers;



First Name	Last Name	Points
Tony	Soprano	450
Maria	Rossi	280
Giuseppe	Bianchi	750
Isabella	Ferrari	120
Marco	Lombardi	890
Sofia	Romano	65

Q33. Create a query showing customer's full name (concatenated) as 'Full Name' and their status as 'Membership Level'.

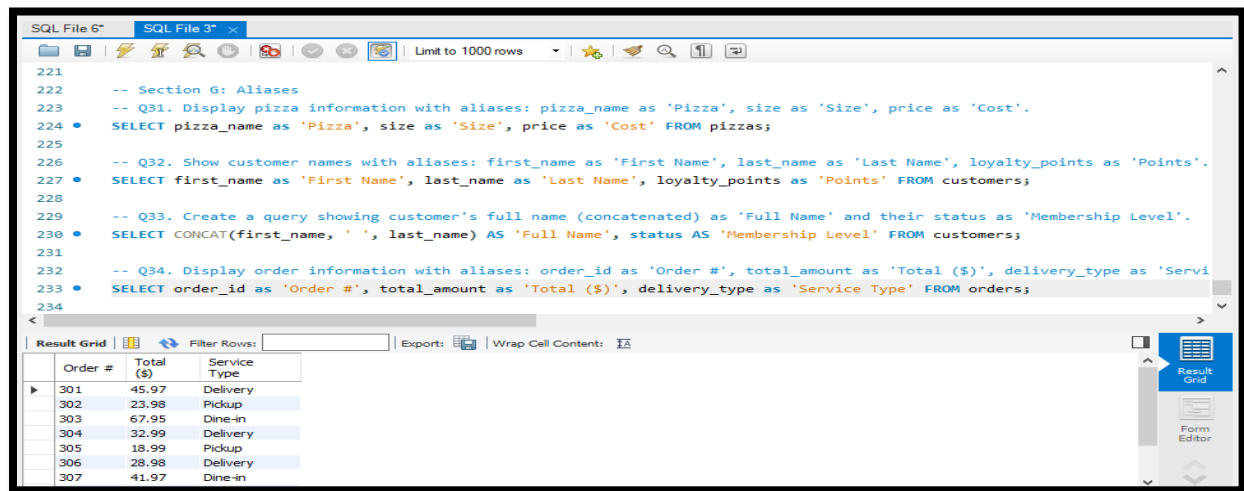
SELECT CONCAT(first_name, ' ', last_name) AS 'Full Name', status AS 'Membership Level' FROM customers;



Full Name	Membership Level
Tony Soprano	VIP
Maria Rossi	Regular
Giuseppe Bianchi	Gold
Isabella Ferrari	Regular
Marco Lombardi	Gold
Sofia Romano	Regular
Antonio Conti	VIP

Q34. Display order information with aliases: order_id as 'Order #', total_amount as 'Total (\$)', delivery_type as 'Service Type'.

SELECT order_id as 'Order #', total_amount as 'Total (\$)', delivery_type as 'Service Type'
FROM orders;



Section H: Combined Operations

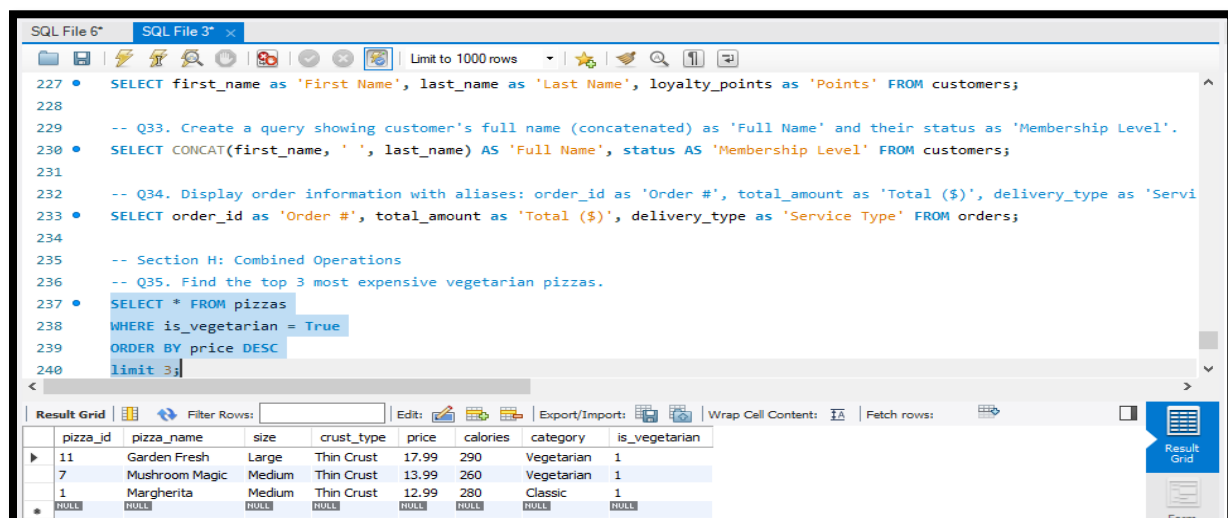
Q35. Find the top 3 most expensive vegetarian pizzas.

SELECT * FROM pizzas

WHERE is_vegetarian = True

ORDER BY price DESC

limit 3;



Q36. Display the first 5 VIP customers ordered by loyalty points (highest first).

```
SELECT * FROM customers
```

```
WHERE status = 'VIP'
```

```
ORDER BY loyalty_points DESC
```

```
limit 5;
```

The screenshot shows a SQL IDE with a query editor and a results grid. The query editor contains the following SQL code:

```
-- Q34. Display order information with aliases: order_id as 'Order #', total_amount as 'Total ($)', delivery_type as 'Service Type' FROM orders;
SELECT order_id as 'Order #', total_amount as 'Total ($)', delivery_type as 'Service Type' FROM orders;

-- Section H: Combined Operations
-- Q35. Find the top 3 most expensive vegetarian pizzas.
SELECT * FROM pizzas
WHERE is_vegetarian = True
ORDER BY price DESC
limit 3;

-- Q36. Display the first 5 VIP customers ordered by loyalty points (highest first).
SELECT * FROM customers
WHERE status = 'VIP'
ORDER BY loyalty_points DESC
```

The results grid displays the following data:

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
201	Tony	Soprano	555-0101	tony.soprano@email.com	Newark	2023-01-15	450	VIP
207	Antonio	Conti	555-0107	antonio.conti@email.com	Staten Island	2022-12-03	340	VIP

Q37. Show Large pizzas under \$20, ordered by calories, limit to 3 results.

```
SELECT * FROM pizzas
```

```
WHERE size ='Large' AND price < 20
```

```
ORDER BY calories ASC
```

```
limit 3;
```

The screenshot shows a SQL IDE with a query editor and a results grid. The query editor contains the following SQL code:

```
SELECT * FROM pizzas
WHERE is_vegetarian = True
ORDER BY price DESC
limit 3;

-- Q36. Display the first 5 VIP customers ordered by loyalty points (highest first).
SELECT * FROM customers
WHERE status = 'VIP'
ORDER BY loyalty_points DESC
limit 5;

-- Q37. Show Large pizzas under $20, ordered by calories, limit to 3 results.
SELECT * FROM pizzas
WHERE size ='Large' AND price < 20
```

The results grid displays the following data:

pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
11	Garden Fresh	Large	Thin Crust	17.99	290	Vegetarian	1
2	Pepperoni Supreme	Large	Hand Tossed	18.99	350	Meat	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0

Q38. Find customers from Brooklyn or Queens with more than 200 loyalty points, ordered by points.

```
SELECT * FROM customers
```

```
WHERE (city = 'Brooklyn' OR city = 'Queens')
```

```
AND loyalty_points > 200
```

```
ORDER BY loyalty_points DESC;
```

The screenshot shows a SQL IDE with a query editor and a results grid. The query editor contains the following SQL code:

```
249 • SELECT * FROM pizzas
250 WHERE size = 'Large' AND price < 20
251 ORDER BY calories ASC
252 limit 3;
253
254 -- Q38. Find customers from Brooklyn or Queens with more than 200 loyalty points, ordered by points.
255 • SELECT * FROM customers
256 WHERE (city = 'Brooklyn' OR city = 'Queens')
257 AND loyalty_points > 200
258 ORDER BY loyalty_points DESC;
259
260 -- Q39. Display the 2nd and 3rd cheapest pizzas in the 'Specialty' category.
261 -- Q40. Show orders from January 17-19, 2024, with delivery type 'Delivery', ordered by total amount.
262
```

The results grid displays the following data:

customer_id	first_name	last_name	phone	email	city	join_date	loyalty_points	status
209	Luca	Esposito	555-0109	luca.esposito@gmail.com	Queens	2022-09-15	520	VIP
202	Maria	Rossi	555-0102	maria.rossi@gmail.com	Brooklyn	2023-02-20	280	Regular

Q39. Display the 2nd and 3rd cheapest pizzas in the 'Specialty' category.

```
SELECT * FROM pizzas
```

```
WHERE category = 'Specialty'
```

```
ORDER BY price ASC
```

```
LIMIT 2 OFFSET 1;
```

The screenshot shows a SQL IDE with a query editor and a results grid. The query editor contains the following SQL code:

```
249 • SELECT * FROM pizzas
250 WHERE size = 'Large' AND price < 20
251 ORDER BY calories ASC
252 limit 3;
253
254 -- Q38. Find customers from Brooklyn or Queens with more than 200 loyalty points, ordered by points.
255 • SELECT * FROM customers
256 WHERE (city = 'Brooklyn' OR city = 'Queens')
257 AND loyalty_points > 200
258 ORDER BY loyalty_points DESC;
259
260 -- Q39. Display the 2nd and 3rd cheapest pizzas in the 'Specialty' category.
261 • SELECT * FROM pizzas
262 WHERE category = 'Specialty'
```

The results grid displays the following data:

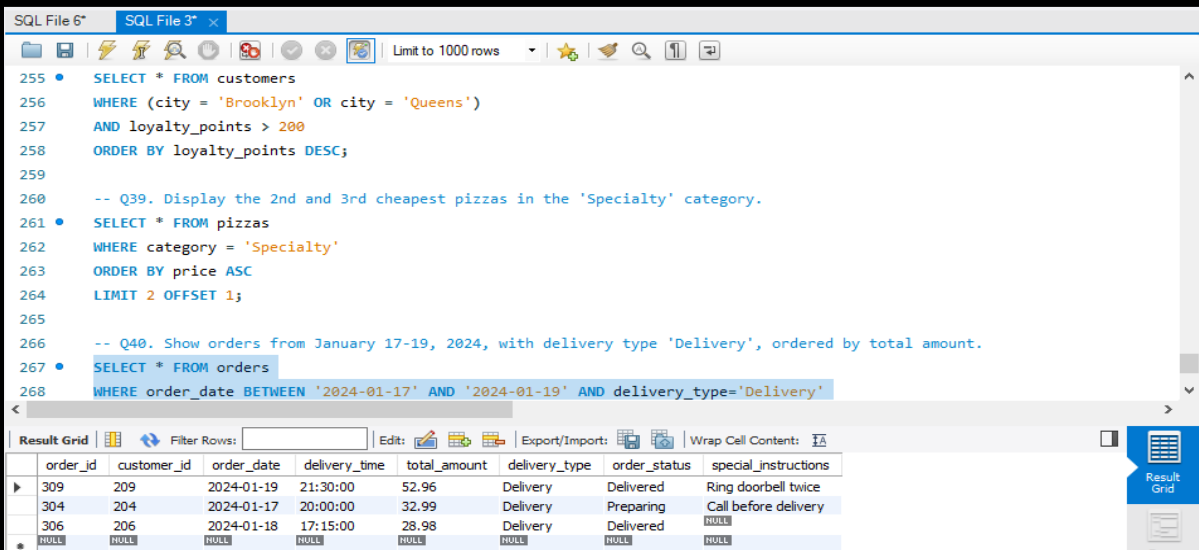
pizza_id	pizza_name	size	crust_type	price	calories	category	is_vegetarian
10	Buffalo Chicken	Medium	Thick Crust	16.99	340	Specialty	0
6	BBQ Chicken	Large	Hand Tossed	19.99	380	Specialty	0

Q40. Show orders from January 17-19, 2024, with delivery type 'Delivery', ordered by total amount.

```
SELECT * FROM orders
```

```
WHERE order_date BETWEEN '2024-01-17' AND '2024-01-19' AND delivery_type='Delivery'
```

```
ORDER BY total_amount DESC;
```



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains several SQL queries, with the last one (Q40) highlighted. The result grid displays the output of the query, showing a table with 8 columns: order_id, customer_id, order_date, delivery_time, total_amount, delivery_type, order_status, and special_instructions. The results are ordered by total_amount in descending order.

```
255 • SELECT * FROM customers
256 WHERE (city = 'Brooklyn' OR city = 'Queens')
257 AND loyalty_points > 200
258 ORDER BY loyalty_points DESC;
259
260 -- Q39. Display the 2nd and 3rd cheapest pizzas in the 'Specialty' category.
261 • SELECT * FROM pizzas
262 WHERE category = 'Specialty'
263 ORDER BY price ASC
264 LIMIT 2 OFFSET 1;
265
266 -- Q40. Show orders from January 17-19, 2024, with delivery type 'Delivery', ordered by total amount.
267 • SELECT * FROM orders
268 WHERE order_date BETWEEN '2024-01-17' AND '2024-01-19' AND delivery_type='Delivery'
```

order_id	customer_id	order_date	delivery_time	total_amount	delivery_type	order_status	special_instructions
309	209	2024-01-19	21:30:00	52.96	Delivery	Delivered	Ring doorbell twice
304	204	2024-01-17	20:00:00	32.99	Delivery	Preparing	Call before delivery
306	206	2024-01-18	17:15:00	28.98	Delivery	Delivered	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL